Fine-grained Parallel Implementation of Post-quantum HQC Key Encapsulation Mechanism on GPUs

Wijden Elmetamri, Mingxiang Chen, Wai-Kong Lee, *Member, IEEE,*, Ramachandra Achar, *Fellow, IEEE,* Seong Oun Hwang, *Senior Member, IEEE.*

***Abstract*—Hamming Quasi-Cyclic (HQC) is an emerging code-based post-quantum Key Encapsulation Mechanism (KEM) standardized by the US National Institute of Standards and Technology (NIST) in 2025. The implementation aspects of HQC are less researched since its inception, unlike its lattice counterparts (e.g., Kyber). This work focuses on the fine-grained parallel implementation of HQC on Graphics Processing Units (GPUs) to achieve high throughput and low latency key encapsulation and decapsulation. The main focus is to optimize the *GF(2)* polynomial multiplication in HQC which takes up ≈90% of the computation time. To overcome this bottleneck, a highly optimized 64-bit carry-less multiplication technique targeting GPU architecture is proposed, effectively improving the throughput of base multiplication in HQC by $2.21\times$. Following this, parallel implementation techniques were proposed to optimize the Karatsuba and Reed-Soloman algorithm on a low-latency fine-grained HQC implementation, which is useful for small batches of workload. This implementation achieved 42172, 26514 and 49384 Encapsulation/s, 25254, 19018 and 35502 Decapsulation/s, on RTX5070, A100 and H100 respectively.**

***Index Terms*—Hamming Quasi-Cyclic, HQC, post-quantum cryptography, polynomial multiplication, GPU.**

I. Introduction

THE emergence of large-scale quantum computing redefines the realm of computational capabilities, posing a direct threat to the wildely adopted cryptographic schemes (e.g., RSA and ECC). Given their critical role in securing communications across digital infrastructures, finance, and industrial control networks, the "harvest now, decrypt later" threat model underscores the urgency of transitioning to quantum-resistant cryptographic standards. US NIST has recently released a series of standardized post-quantum schemes that can withstand such threats, in which HQC [1] was the most recent one. Prior works proposed optimized implementations for Kyber [2], Falcon [3] and SPHINCS+ [4] allowing high performance KEM and signature to be generated efficiently.

In this paper, we present the first parallel fine-grained GPU implementation of HQC KEM on various state-of-the-art GPU architectures. Our contributions are summarized below:

1) The most time-consuming computation in HQC is the *GF(2)* polynomial multiplication, which boils down to the 64-bit carry-less base multiplication. To overcome this bottleneck, a highly optimized 64-bit carry-less multiplication technique targeting GPU architecture is proposed, effectively improving the base multiplication throughput in HQC by $2.21\times$.
2) The fine-grained parallel implementation of HQC targeting low-latency performance is proposed, which is particularly useful in the precense of small to medium workload. Key contributions are optimized parallel implementation of the Karatsuba, Reed-Muller and Reed-Soloman algorithms, facilitating HQC implementations that achieved high throughput performances on GPU architectures with batch sizes of as small as 256 – 1024.
3) The source code of base multiplication is shared in the public domain: https://github.com/benlwk/hqc-fine.git/

The proposed low-latency HQC KEM implementation is particularly useful in IoT applications. It demonstrates the capability to fully utilize the GPU resources with small batches of workload, typically found in the IoT gateway devices that only connects to hundreds of sensor nodes.

II. Background

A. HQC Specifications

HQC is a code-based post-quantum cryptographic scheme whose security is based on the assumed hardness of decoding random linear codes in the Hamming metric. It is defined over a decodable concatenated code C of length $n_1 n_2$ and dimension k, instantiated using Reed–Solomon codes as the outer code and Reed–Muller codes as the inner code. All computations are carried out over the vector space $\mathbb{F}_2^n$, where n denotes the smallest primitive prime greater than $n_1 n_2$. When required, the excess $n - n_1 n_2$ bits are truncated according to the specification.

B. Galois Field (GF(2)) Polynomial Multiplication

Polynomial multiplication in HQC is essentially a carry-less multiplication applied over large polynomials. The large polynomial is divided into multiple chunks of smaller polynomials with each of 64-bit size. For instance, HQC-1 polynomial with length 17,669 is divided into 276 polynomials with 64 bits each and one polynomial with 5 bits. This facilitates the subsequent computations using 64-bit word size, which is denoted as "base_mul" in the subsequent part of the manuscript. Note that HQC is using *GF(2)* polynomials, each coefficient having a binary value, where multiplication between two polynomials involve XOR only operations without carry.

Wijden Elmetamri is with the Institut National de la Recherche Scientifique (INRS), Centre Énergie Matériaux Télécommunications, Canada (e-mail: wijden.elmetamri@inrs.ca).

Mingxiang Chen and Ramachandra Achar are with the Department of Electronics Engineering, Carleton University, Canada. (e-mail: ediechen@cmail.carleton.ca, achar@doe.carleton.ca)

Wai-Kong Lee is with the Faculty of Information and Communication Technology, Universiti Tunku Abdul Rahman, Malaysia (e-mail: wklee@utar.edu.my).

Seong Oun Hwang is with the Department of Computer Engineering, Gachon University, Seongnam 13120, South Korea. (e-mail: sohwang@gachon.ac.kr

Manuscript received ; revised .

Algorithm 1 BASEMUL1BIT_GPU: 1-bit by 1-bit base_mult

Require: 64-bit integers a, b
Ensure: 128-bit result $c = (c_1, c_0)$
1: $z_0 \leftarrow 0,\ z_1 \leftarrow 0$
2: **for** $i = 0$ **to** 63 **do**
3: $z_0 \leftarrow a \times (b\ \&\ 1)$
4: **if** $i > 0$ **then**
5: $z_1 \leftarrow z_0 \gg (64 - i)$
6: **end if**
7: $z_0 \leftarrow z_0 \ll i$
8: result_lo $\leftarrow$ result_lo $\oplus\ z_0$
9: result_hi $\leftarrow$ result_hi $\oplus\ z_1$
10: $b \leftarrow b \gg 1$
11: **end for**
12: $c[0] \leftarrow$ result_lo ▷ Lower 64 bits
13: $c[1] \leftarrow$ result_hi ▷ Higher 64 bits

III. PROPOSED OPTIMIZED 64-BIT CARRY-LESS MULTIPLICATION

To be noted that the HQC reference implementation utilizes the carry-less "base_mul" algorithm `mul1` from [5], which is not optimal for GPU due to large window size that translates to excessive number of registers used. In this work, we present an improvement for the "base_mul" algorithm targeting GPU architectures. Referring to Algorithm 1, a naive implementation of carry-less multiplication scans the multiplicand *a* bit-by-bit (line 2) and multiply it with multiplier *b* (lines 8 and 9) if a particular bit is '1' (line 4). Proposed Algorithm 2 shows an improved version that scans 2-bit by 2-bit, effectively reducing the **for** loop branching overhead by half. Note that lines 2 and 3 only need to precompute powers of 2 due to binary arithmetic involved. Following the similar approach, we can derive 4-bit, 8-bit and 16-bit version accordingly, but increasing the window size may cause excessive usage of registers, which is a scarce resource on a GPU. An ablation study was carried out (see Section IV) to choose the best window size suitable for the GPU architectures.

IV. FINE-GRAINED HQC IMPLEMENTATION

A. Fine-Grained Parallel Implementation of HQC

This work follows the prior fine-grained parallel implementation of signature [3], in which *K* blocks are launched in parallel, each block computes one encapsulation/decapsulation utilizing *T* threads. Note that *K* corresponds to the batch size of workload, while *T* depends on the parallelism available, which can be different for each operation. We have implemented HQC-128 on RTX 5070, A100 and H100 GPU.

B. Parallel Implementation of Karatsuba Algorithm

The Karatsuba algorithm reduces the complexity of large polynomial multiplications by splitting the large polynomials into two smaller ones, trading expensive multiplications for cheaper additions. Note that multiplication on smaller polynomials yields faster performance but consumes more memory. This technique can be recursively applied until the polynomials cannot be further divided, in the expense of decreasing parallelism. We only perform two levels of recursion to allow high parallelism and to prevent thread divergence issues.

Karatsuba algorithm does not have any data dependency between each polynomial coefficient, thus allowing fine-grained parallelism to be mapped efficiently onto GPU threads. Referring to the proposed Algorithm 3, the split process (lines 5 – 8) breaks the two large polynomials into half. These two polynomial multiplications are computed using schoolbook method (lines 9 and 10) to generate $Z(x)_1$ and $Z(x)_1$, wherein each thread calculates one coefficient. The auxiliary operands are computed (line 11) where additions represent GF(2) XORs, and the resultant polynomials are then multiplied to generate $Z(x)_1$. This reconstruction step is also embarrassingly parallel, in which *T* threads within a block cooperatively complete the computation.

Algorithm 2 BASEMUL2BIT_GPU: 2-bit by 2-bit base_mult

Require: 64-bit integers a, b
Ensure: 128-bit result $c = (c_1, c_0)$
1: **Precompute** 128-bit powers of 2:
2: pow_lo[0] $\leftarrow a,$ pow_hi[0] $\leftarrow 0$ ▷ $a \ll 0$
3: pow_lo[1] $\leftarrow a \ll 1,$ pow_hi[1] $\leftarrow a \gg 63$ ▷ $a \ll 1$
4: lo $\leftarrow 0,$ hi $\leftarrow 0$
5: **for** chunk $= 0$ **to** 31 **do**
6: idx $\leftarrow b\ \&\ 3$ ▷ Extract 2-bit
7: $x_{\text{lo}} \leftarrow$ pow_lo[0] & $-$ (idx & 1) $\oplus$ pow_lo[1] & $-$ (idx $\gg$ 1 & 1)
8: $x_{\text{hi}} \leftarrow$ pow_hi[0] & $-$ (idx & 1) $\oplus$ pow_hi[1] & $-$ (idx $\gg$ 1 & 1)
9: $s \leftarrow$ chunk $\ll 1$ ▷ Shift: 0,2,4,...,62
10: lo $\leftarrow$ lo $\oplus\ (x_{\text{lo}} \ll s)$
11: **if** $s > 0$ **then**
12: lo $\leftarrow$ lo $\oplus\ (x_{\text{hi}} \gg (64 - s))$
13: **end if**
14: hi $\leftarrow$ hi $\oplus\ (x_{\text{hi}} \ll s)$
15: **if** $s > 0$ **then**
16: hi $\leftarrow$ hi $\oplus\ (x_{\text{lo}} \gg (64 - s))$
17: **end if**
18: $b \leftarrow b \gg 2$
19: **end for**
20: $c[0] \leftarrow$ lo, $c[1] \leftarrow$ hi

Note that our work is fundamentally different from [6], [7]; we parallelize Karatsuba across many threads targeting lower latency, while [6], [7] pursue the coarse-grained approach with one or three threads only, aiming for high throughput.

C. Parallel Implementation of Reed-Solomon and Reed-Muller

1) Reed-Solomon Encoding: The central parallelization challenge is the LFSR feedback recurrence, which is strictly sequential across 16 iterations. This is resolved by isolating the scalar gate computation and shift-register update on a single lane, while mapping the 31 independent GF(2^8) multiplications per iteration onto a full warp, linked by a `__shfl_sync` register broadcast that eliminates any shared memory round-trip. The kernel is launched with *K* blocks and 256 threads per block. Figure 1 illustrates the four-phase thread pipeline within each block.

Algorithm 3 Karatsuba Multiplication

Require: Two polynomials $a(x)$, $b(x)$, base B
Ensure: $a(x) \times b(x)$

function KARATSUBA($a(x)$, $b(x)$)
  if $a(x) < B$ **or** $b(x) < B$ **then**
    return $a(x) \times b(x)$
  end if
  $m \leftarrow \max(\text{size\_base}(a(x)), \text{size\_base}(b(x)))$
  $m_2 \leftarrow \lfloor m/2 \rfloor$
  $a(x)_1, a(x)_0 \leftarrow \text{split\_at}(a(x), m_2)$
  $b(x)_1, b(x)_0 \leftarrow \text{split\_at}(b(x), m_2)$
  $z(x)_0 \leftarrow karatsuba(a(x)_0, b(x)_0)$
  $z(x)_2 \leftarrow karatsuba(a(x)_1, b(x)_1)$
  $z(x)_1 \leftarrow karatsuba(a(x)_0 + a(x)_1, b(x)_0 + b(x)_1) - z(x)_0 - z(x)_2$
  return $z(x)_2 \cdot B^{2m_2} + z(x)_1 \cdot B^{m_2} + z(x)_0$
end function

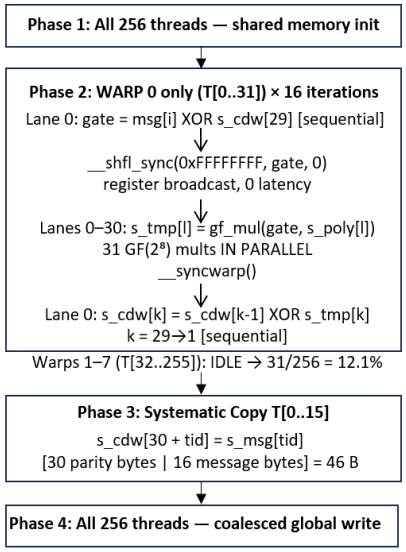

Fig. 1. Reed-Solomon Encoding Thread Pipeline

The `__shfl_sync` delivers the gate value register-to-register within a single clock, enabling 31 parallel GF multiplications without any synchronization barrier. The sequential shift-register update in lane 0 remains the irreducible bottleneck, strictly bounded by the LFSR data dependency chain.

2) Reed-Solomon Decoding: The Berlekamp-Massey-Forney pipeline is distributed across five GPU kernels. The parallelization strategy is heterogeneous by design: position-independent stages exploit full thread-by-batch parallelism, while the BM recurrence is restricted to single-thread execution per block, with batch-level SM concurrency as the only available parallelism. Figure 2 summarizes the structure, grid configurations, and thread utilization across all five kernels.

Kernels K2 through K4 allocate 256 threads despite only one being active per block. This is intentional: it maintains uniform SM occupancy across the pipeline and avoids reconfiguration overhead between kernel launches. The BM state arrays `sigma_copy` and `X_ sigma_v` are held in shared

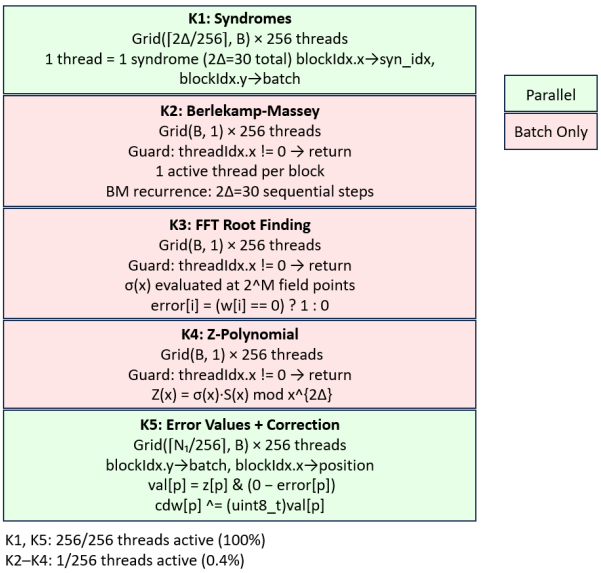

Fig. 2. Reed-Solomon Decoding: 5-Kernel Pipeline

memory across all 30 iterations to prevent eviction to L2. The branchless mask `(0 - error[pos])` used in K5 preserves timing uniformity, which is a necessary property in a post-quantum cryptographic context.

3) Reed-Muller Encoding: This kernel is launched with one block per sample and 256 threads per block. Since all 46 symbol encoding are data-independent, threads distribute the work via a stride loop. Given that `VEC_N1_SIZE_BYTES` $= 46 < 256$, threads 0 through 45 each handles exactly one byte, and no synchronization is needed at any point within the block. The `encode_rm()` device function runs entirely in registers, encoding 8 bits into a 128-bit codeword through bitmask XOR accumulation over the RM(1,7) generator rows.

4) Reed-Muller Decoding: This kernel is launched with a 64-thread block size, where each thread maps exactly to one of the 64 linear functionals required by the RM(1,7) decoder.

Stage A: Linear Functional Evaluation (T[0..63]). Each thread independently evaluates a distinct linear functional over the full `VEC_N1N2_SIZE_64`-word received vector. Thread `tid` constructs a 64-bit mask per input word where bit j is set if `(pos >> tid) & 1`, then accumulates `__popcll(word & mask)`. All 64 functionals are evaluated simultaneously with no inter-thread communication, and results are deposited into shared array `sums[64]` before a single synchronization `__syncthreads()`.

Stage B: Majority Vote (T[0] only). Thread 0 reads `sums[63]` and compares it against the threshold `PARAM_N1N2/4` $= 4416$, writing 8 output bytes as either `0xFF` or `0x00`. Although all 64 entries of `sums[]` are computed, only index 63 (`0b0111111`) carries the information bit in the RM(1,7) structure.

V. Experimental Results and Discussions

A. Micro-benchmarking 64-bit Base Multiplication

Table I shows the throughput of "base_mul" implementation following the proposed optimized 64-bit carryless multiplica-

TABLE I
PERFORMANCE PROPOSED VARIANTS OF BASE_MUL ON RTX 5070

Batch (K)	base_mul [1]	1-bit	2-bit	4-bit	8-bit
	Multiplication per second				
128	9864	13020	13038	20259	20203
2048	12217	19643	22821	24735	23411
8192	12328	**24649**	27192	**24763**	24127
16384	**12353**	24628	27375	24607	**24262**
32768	12297	24548	27376	24609	24213

TABLE II
THROUGHPUT (OPS/SECOND) OF HQC EN/DECAP ON DIFFERENT GPUS

Batch (K)	RTX 5070 2325 MHz 6144 cores	A100 765 MHz 6912 cores	H100 1095 MHz 14592 cores
	Operation per second*		
64	14745 (18920)	3007 (5796)	5966 (14241)
256	30231 (22799)	10622 (12019)	18914 (26990)
1024	38614 (24537)	16764 (14891)	33448 (30614)
4096	41057 (25252)	23976 (18342)	40583 (31485)
8192	42046 (25241)	24106 (18551)	41843 (31998)
16384	42126 (25253)	25110 (18758)	43726 (32286)
32768	42160 (**25254**)	25275 (18797)	40171 (32459)
60000	**42172** (25240)	25404 (18867)	44614 (34535)
81920	–	**26514** (**19018**)	**49384** (**35502**)
Intel Core-i7-11850H CPU at 2.50GH [1]			274 (180)
HIGH, RTX 4090, 16384 cores, 2.23 GHz clock [7]			586560 (374160)

*Values in brackets denote decapsulation throughput.

tion. Experimental results show that the proposed technique is faster than the original implementation, with the best throughput achieved by 2-bit window size at batch size 4096 (27397). This is $2.21\times$ faster than the best throughput achieved by [1] (12353). Based on this experimental result, we use the 2-bit window size for implementing the carryless polynomial multiplication in HQC encapsulation and decapsulation.

B. HQC Performance on Different GPUs

Referring to Table II, our fine-grained GPU implementation of HQC achieved the highest throughput at 42172, 26514 and 49384 Encapsulation/s, 25254, 19018 and 35502 Decapsulation/s, on RTX5070, A100 and H100 respectively. When the batch size K increases, the throughput also increases almost linearly across all GPU architectures. A closer look into the results shows that our proposed GPU implementation already can achieve performance close to its maximum throughput for batch size between 1024 – 4096. This shows that fine-grained approach can fully exploit the GPU resources even with small batch sizes. Our work on RTX 5070 is also $154\times$ and $234\times$ faster than the CPU implementation [1].

C. Comparison With Other HQC Implementation

HIGH [7] achieved impressive throughput performance, but their coarse-grained implementation is not open source. We try to replicate their work (UKarat3) closely. Table III shows the latency achieved by the proposed fine-grained implementation compared to the coarse-grained version, which is a replication of [7]. Our fine-grained approach can achieve higher throughput and lower latency at small batch sizes ($\leq$512) compared to coarse-grained version. When the batch is larger than 512, coarse-grained implementation would be the clear winner.

TABLE III
COMPARISON WITH COARSE-GRAINED HQC ON RTX 5070

Batch (K)	Latency (ms)		TP (Op/s)
	Fine	Coarse	Coarse
4	2.28 (1.04)	14.58 (27.29)	274 (147)
64	3.04 (3.06)	15.36 (29.07)	4165 (2202)
256	5.86 (9.65)	15.26 (29.66)	16776 (8633)
512	10.70 (18.65)	16.66 (30.04)	37975 (21992)
1024	20.70 (38.65)	17.66 (32.78)	57975 (32992)
4096	76.73 (146.51)	29.16 (45.81)	140463 (89419)
16384	314.26 (580.26)	87.03 (124.51)	188242 (131597)

*Values in brackets denote decapsulation throughput.

VI. CONCLUSION

The proposed fine-grained GPU implementation of HQC can achieve high throughput performance even at a batch size as low as 512. This indicates that our work can effectively exploit the GPU resources without requiring a large amount of work. Such achievement can be utilized on applications that do not typically have large batch size. For example, IoT gateways, vehicle-to-everything (V2X) and edge AI inference. Furthermore, the fine-grained kernel can be integrated alongside a coarse-grained implementation within a heterogeneous scheduling framework, enabling dynamic workload-aware performance scaling across varying concurrency levels.

ACKNOWLEDGEMENT

Wai-Kong Lee and Seong Oun Hwang were supported by the Korea NRF grant funded by the Korea government (MSIT) (RS-2024-00340882). Ramachandra Achar and his group at Carleton University is supported by the discovery grant from the Natural Science and Engineering Research of Canada (NSERC) Canada.

REFERENCES

[1] C. A. Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, E. Persichetti, G. Zémor, and I. Bourges, "Hamming quasi-cyclic (hqc)," *NIST PQC Round*, vol. 2, no. 4, p. 13, 2018.

[2] X. Ji, J. Dong, T. Deng, P. Zhang, J. Hua, and F. Xiao, "Hi-kyber: A novel high-performance implementation scheme of kyber based on gpu," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 6, pp. 877–891, 2024.

[3] W.-K. Lee, R. K. Zhao, R. Steinfeld, A. Sakzad, and S. O. Hwang, "High throughput lattice-based signatures on gpus: Comparing falcon and mitaka," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 4, pp. 675–692, 2024.

[4] D. Kim, H. Choi, and S. C. Seo, "Parallel implementation of sphincs+ with gpus," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 6, pp. 2810–2823, 2024.

[5] R. P. Brent, P. Gaudry, E. Thomé, and P. Zimmermann, "Faster multiplication in gf (2)[x]," in *International Algorithmic Number Theory Symposium*. Springer, 2008, pp. 153–166.

[6] J. Dong, Y. Fu, X. Qin, Z. Dong, F. Xiao, and J. Lin, "Eco-bike: Bridging the gap between pqc bike and gpu acceleration," *IEEE Transactions on Information Forensics and Security*, 2024.

[7] J. Dong, Y. Hou, S. Wang, L. Sha, F. Xiao, Z. Dong, and J. Lin, "High: Harnessing gpu parallelism for optimized hqc performance," *Cryptology ePrint Archive*, 2026.